RESEARCH PAPER TECHNICAL RESOURCE & SYSTEM SPECIFICATION

SkillPlus AI: Full Operational Workflow, Global Backend Architecture, and Feature-by-Feature Algorithmic Logic

A Comprehensive Academic Blueprint Covering End-to-End User Journeys, Serverless ASGI Execution, Multi-Modal Career Intelligence, Forensic Anti-AI Skill Diagnostics, and Dynamic Execution Sprints

Target System: SkillPlus AI (SkillGap AI Ecosystem)	**Core Architecture:** Decoupled React 18 SPA + FastAPI Serverless ASGI
Deployment Topology: Unified Single-Origin on Vercel Edge	**Inference Engines:** Groq LPU (Llama-3.3-70B), Procedural Synthesis, Heuristic Engine
Document Purpose: Research Paper System Methodology & Reference	**Data Invariants:** Zero Mock Data Policy, Zero Disk Leaks, Privacy-Preserving Streams

Abstract

Modern developer evaluation suffers from a critical epistemological crisis: the widespread adoption of generative AI tools enables candidates to produce polished, syntactically complex code repositories and keyword-stuffed professional resumes without possessing authentic engineering comprehension. Traditional Applicant Tracking Systems (ATS) and star-based GitHub heuristics fail to distinguish genuine engineering mastery from prompted assembly. This paper presents **SkillPlus AI** (also operationalized as SkillGap AI), a full-stack career intelligence architecture engineered to evaluate, verify, and upskill modern technical talent. We systematically detail the platform across three foundational dimensions: (1) **The Whole Flow of the Page**, specifying the complete client-side journey, interactive state machines, visual feedback loops, and sequential user progression; (2) **Global Backend Architecture & System Infrastructure**, delineating the serverless ASGI lifecycle, single-origin edge rewrites, resilient multi-tier persistence (RAM, /tmp, and cloud relational databases), and security protocols; and (3) **Feature-by-Feature Operational Workflows & Respective Backend Engines**, dissecting the mathematical formulations, regex segmentation pipelines, multi-threaded GitHub file-tree AST-free inspections, dual-engine non-predefined quiz synthesis, Bayesian-heuristic authenticity scoring, multi-source roadmap merging, and Groq LPU context-grounded conversational reasoning. By strictly focusing on operational workflows and algorithmic logic without code syntax clutter, this document serves as a self-contained formal methodology resource for software engineering and educational computing research.

Keywords: *Career Intelligence, Anti-AI Skill Verification, Code Authenticity, Resume PDF Parsing, GitHub Forensic Analysis, Dynamic Roadmaps, Vercel Serverless ASGI, Groq LPU Inference, Heuristic Scoring Models.*

PART 1: The Whole Flow of the Page (Complete User Journey & Frontend Workflow)

The SkillPlus AI frontend is organized as a high-performance Single Page Application (SPA) designed to guide the candidate through an intuitive, progressive disclosure workflow. The user is transitioned sequentially from initial discovery, through multi-modal profile ingestion, into active technical verification, and ultimately to personalized career roadmapping and interactive coaching.

1.1 Lifecycle Initialization & Intro Animation Phase

Upon visiting the web application, the application checks browser session memory to determine if the candidate is a first-time or returning visitor. For new sessions, an immersive cinematic Intro Animation plays full-screen, establishing the high-tech, cyber-brutalist aesthetic of the platform. The sequence automatically times out or can be dismissed via an instant skip trigger. As the intro concludes, the global layout shell unhides, and manual scroll restoration resets the viewport precisely to the top of the interface.

1.2 Navigation Bar & ScrollSpy Dynamics

The top navigation bar acts as a persistent command deck. It houses the platform logo, system status indicators (confirming live backend connectivity), and direct jump anchors corresponding to the seven key functional sections: Home, LinkedIn Analyzer, GitHub Analyzer, AI Skill Verifier, Career Roadmap, AI Career Coach, and Frequently Asked Questions. A custom ScrollSpy hook continuously monitors the user's scroll position, updating the active navigation pill dynamically as different sections enter the primary viewport.

1.3 Interactive Hero Section & Visual Metrics Banner

The Hero section presents the value proposition: 'Close Your Skill Gap with AI'. It features responsive parallax geometric shapes that shift in subtle opposition to mouse cursor coordinates, reinforcing tactile interactivity. Prominent Call-to-Action (CTA) buttons ('Analyze LinkedIn →' and 'Analyze GitHub ■') provide one-click smooth scrolling directly to their respective ingestion modules. A visual metric showcase highlights simulated aggregate benchmarks (Overall Score, Detected Gaps, and Authenticity Badges) to set clear expectations of the analytical outputs.

1.4 Dual-Direction Continuous Marquee Dividers

Positioned immediately below the Hero and Bento sections, dual high-speed text marquees scroll continuously in opposite directions. These ticker ribbons display active platform competencies, including 'LinkedIn Profile Analyzer', 'GitHub Code Review', 'Anti-AI Code Verifier', and 'Market-Aligned Sprint Planning', creating clear visual boundaries between thematic page sections.

1.5 Interactive Bento Feature Grid & Ambient Eye-Tracking

The Bento Grid showcases the primary architectural modules inside distinct high-contrast cards. Each card features an animated ocular pupil mechanism. When the candidate moves their pointer across the screen, the pupils compute trigonometric angles relative to their card boundaries and follow the cursor. During periods of user inactivity, the eyes enter an autonomous ambient roaming state, drifting between predetermined screen waypoints and executing randomized blinks. Clicking any Bento card triggers an animated smooth scroll directly to the corresponding functional engine.

1.6 LinkedIn Profile Analyzer Section: PDF Drag-and-Drop Experience

The user reaches the LinkedIn Analyzer section. If no profile has been analyzed yet, the interface displays an illustrated 4-step instructional guide explaining how to export an official profile PDF directly from LinkedIn's native 'More → Save to PDF' button. Directly beneath the guide sits an active drag-and-drop upload zone supporting PDF files up to 15MB. The candidate can drag their file into the target zone or browse locally. Once a PDF is selected, the file size and filename are validated, and the candidate clicks 'Analyze LinkedIn Profile Online (Free)'. A pulsing loading state is initiated. Upon backend resolution, the upload form collapses and is replaced by a comprehensive, color-coded **Candidate Profile Dossier** detailing their verified name, headline, 3-line executive summary, clickable contact pills, structured work experience timeline, academic degrees, detected technical skills, and certified licenses, accompanied by a 'Upload Another PDF' reset button.

1.7 GitHub Repository Analyzer Section: Real-Time Autocomplete & Inspection

The candidate progresses to the GitHub Analyzer section. The input field features an intelligent debounced autocomplete system. As the candidate begins typing a username or pasting a GitHub profile link, the system queries the backend to provide real-time suggestions featuring avatars, account names, and user bios. Upon selecting their username or submitting the form, the candidate triggers an in-depth repository review. The UI displays an animated loader while the backend performs multi-threaded repository inspections. Once complete, the view populates with: (1) an overall Developer Review Score badge; (2) aggregate metric cards (public repositories, cumulative stars, followers, and an activity index); (3) primary language share progress bars with exact percentage distributions; (4) a prominent 'Start Test for Primary Language' verification trigger; and (5) cards showcasing the candidate's top starred repositories.

1.8 AI Code & Skill Authenticity Verifier Section: Adaptive Diagnostic Quiz

The Skill Verifier is conditionally unlocked once either GitHub or LinkedIn analysis has occurred, or immediately if the candidate clicks 'Verify' next to any detected language. In the initial setup state, the engine automatically selects the candidate's strongest verified programming language extracted from their GitHub repositories, while providing dropdown options to select alternative languages (Python, JavaScript, TypeScript, C, C++, Java, Go, Rust, HTML). Clicking 'Start Assessment' generates a custom, non-predefined 10-question diagnostic exam. The view shifts into an active test environment containing: a question counter (1 to 10), difficulty badges (Fundamental, Intermediate, Deep Mechanics), concept tags, contextual code snippets referencing the candidate's real GitHub repositories, and four selectable radio options. The candidate navigates questions, selects choices, and submits the test. The client records active time spent and transmits answers to the backend. The test view immediately updates to an **Authenticity Scorecard** displaying: the final score (e.g. 8/10, 80%), a formal authenticity verdict ('Strong Proficiency Verified', 'Developing / Practice Recommended', or 'Skill Gap Identified / Fundamentals Review'), a diagnostic concept-by-concept rating matrix, detailed question reviews showing exact correct answers and explanations, and actionable remediation guidance.

1.9 Unified 4-Week Adaptive Roadmap Section: Synthesized Sprint Plans

Next, the candidate arrives at the Roadmap section. If either LinkedIn or GitHub data is cached, status indicators confirm ingestion and activate the 'Build My Roadmap' action. Clicking this synthesizes qualitative resume gaps with empirical repository deficiencies. The generated roadmap presents: (1) a Combined Career Readiness Score (e.g. 84/100) with a qualitative executive summary; (2) metric badges indicating primary stack focus, sprint count, and timeline; and (3) four progressive sprint milestone cards (Week 1 to Week 4). Each card outlines specific themes (e.g. Test Suite Implementation, CI/CD Pipeline Automation, Containerization, System Architecture), key tangible deliverables, and interactive task checklists with checkboxes that candidates can mark completed during their study journey.

1.10 Context-Injected AI Career Coach Chatbot Section

The Career Coach provides continuous conversational guidance. The chat widget displays a contextual identity tag showing the active candidate's name and GitHub handle. Below the conversational stream, dynamic suggestion pills adapt to the candidate's state: if profiles have been analyzed, pills provide instant queries like 'Review my overall profile', 'How is my GitHub profile?', or 'Explain my dynamic roadmap'; if no profile is analyzed, general pills like 'How does SkillPlus AI work?' or 'Why is System Design important?' appear. Candidates can also type custom inquiries. The conversational stream updates with sub-second latency via Groq LPU inference or local reasoning engines.

1.11 Comprehensive FAQ Accordion & Semantic Footer

The page concludes with an interactive brutalist FAQ accordion addressing security, zero-disk data handling, ATS compatibility, and scoring models, followed by a semantic footer containing quick jump anchors and architectural copyright metadata.

FRONTEND ARCHITECTURAL PRINCIPLE 01: Client State Management Flow: The frontend coordinates all module interactions through a centralized React Context (AnalysisContext). When a LinkedIn PDF or GitHub profile resolves, their respective result objects (linkedinResult, githubResult) are saved in global state. This state automatically unlocks downstream modules (Verifier and Roadmap), auto-selects the primary programming language, invalidates stale roadmaps, and injects full user context into the floating AI Career Coach.

PART 2: Global Backend Architecture & System Infrastructure Flow

The SkillPlus AI backend is architected as an asynchronous, modular micro-service monolith built on Python 3.12 and FastAPI. It is engineered specifically to operate under the strict memory and filesystem boundaries of serverless edge execution while delivering sub-second response times.

2.1 Serverless ASGI Request Flow & Unified Single-Origin Pattern

Traditional full-stack deployments separate the frontend application (e.g. on Vercel or Netlify) from the backend API (e.g. on AWS EC2, Heroku, or Render). This decoupling introduces cross-origin latency, CORS preflight negotiation overhead, and synchronization fragility. SkillPlus AI solves this via a **Unified Single-Origin Serverless Topology** governed by Vercel edge configuration:

- **Vercel Edge Rewrite Engine:** All incoming HTTP requests to the domain are intercepted by edge rewrite rules defined in *vercel.json*, redirecting all paths to the ASGI handler at *api/index.py*.
- **ASGI Entrypoint Resolution (*api/index.py*):** The entrypoint dynamically injects the root and backend source directories into the Python runtime system path (*sys.path*), instantiating the core FastAPI application object.
- **Three-Way Routing Dispatcher:** The FastAPI application inspects incoming request URLs and delegates execution through three paths: (1) Requests matching */api/** are routed to modular API routers; (2) Requests matching static asset paths (*/assets/**) are served directly from the compiled static production bundle; and (3) All non-API client routes (*/**) return the pre-rendered single-page entry document (*index.html*).

2.2 Multi-Tier Persistence & Serverless Ephemeral Disk Safety

Serverless functions operate on strictly read-only file systems (*/var/task*). Any attempt to write local state, cache tokens, or session databases directly to the codebase directory results in a fatal operating system exception (*OSError*). SkillPlus AI resolves this through a three-tier resilient persistence hierarchy:

- **Tier 1: In-Memory Active Cache:** Python memory dictionaries maintain active LinkedIn analyses, GitHub results, active quiz sessions, and chat transcripts for instantaneous, sub-millisecond retrieval during the function container's warm lifecycle.
- **Tier 2: Ephemeral Disk Fallback:** The storage engine checks environment variables at runtime. If *os.environ.get('VERCEL')* is detected, all disk write attempts are routed exclusively to the operating system's temporary directory (*/tmp/skillgap_data*). All disk write operations are wrapped in non-blocking defensive exception handlers.
- **Tier 3: Distributed Cloud Persistence (Supabase Integration):** When cloud credentials are configured, the persistence service asynchronously mirrors records into external PostgreSQL tables (*linkedin_results*, *github_results*, *roadmap_results*, *quiz_sessions*, *chat_history*), enabling persistent multi-session analytics across cold container restarts.

2.3 Cross-Origin Security & Request Validation Engine

The backend implements CORS middleware with dynamic origin whitelisting, permitting local development ports (*localhost:5173*, *localhost:8000*) while enforcing strict access policies for production domains. All payloads are validated using Pydantic schemas. Custom exception handlers intercept validation errors and HTTP exceptions, transforming low-level stack traces into standardized, client-friendly JSON error responses.

HTTP Method	Endpoint Route	Subsystem Router	Functional Responsibility
GET	/api/health	Core System	Health check & active modular router status
POST	/api/linkedin/analyze-pdf	LinkedIn Analyzer	In-memory PDF text extraction & dossier synthesis
POST	/api/github/analyze	GitHub Analyzer	Multi-repo AST-free hygiene & language audit
GET	/api/github/search-users	GitHub Analyzer	Debounced developer search with TTL cache
POST	/api/skill-verifier/generate	Skill Verifier	Dual-engine non-predefined 10Q diagnostic generation
POST	/api/skill-verifier/submit	Skill Verifier	Authenticity Index scoring & concept evaluation
POST	/api/roadmap/build	Roadmap Engine	Cross-analyzer synthesis & 4-week sprint creation

POST	/api/chat/message	AI Career Agent	Groq LPU / Heuristic context-injected career chat

PART 3: Feature-by-Feature Operational Workflows & Respective Backend Engines

This section presents a rigorous, dual-sided analysis of every individual capability in SkillPlus AI. For each feature, we first trace the complete frontend interaction workflow and UI state machine, followed immediately by the exact mathematical formulas, data transformations, and algorithmic execution logic powering its backend engine.

3.1 Feature 1: LinkedIn Profile & PDF Intelligence Analyzer

Frontend User Workflow & Interaction Logic:

1. The candidate accesses the LinkedIn Analyzer module, where an instructional step-by-step card illustrates how to download the official profile PDF.
2. The candidate drags their downloaded PDF into the brutalist dropzone or selects it via the file browser dialog.
3. The client verifies that the selected file possesses a valid .pdf MIME extension and does not exceed file size constraints.
4. Submitting the form triggers a multi-part form data upload to the backend, rendering an animated processing indicator.
5. When the backend returns the parsed dossier, the client updates the global AnalysisContext, hides the upload form, and renders the verified Candidate Dossier (personal identity, professional summary, contact pills, experience timeline, academic degrees, and skills).
6. An 'Upload Another PDF' button enables resetting the state to analyze alternative candidate documents.

Backend Architecture, Data Pipeline, & Algorithmic Logic:

The backend LinkedIn engine processes documents entirely in-memory through a multi-stage deterministic pipeline:
• **In-Memory Byte Ingestion:** Incoming PDF bytes are wrapped in an in-memory byte stream and parsed using an asynchronous PDF reader. Plain text is extracted across all document pages without writing temporary files to disk.
• **Contact Shield & Sanitization Algorithm:** Profile PDFs contain numerous numerical sequences (vanity URL slugs, graduation years, and postal codes) that frequently cause false-positive phone matches. The backend applies a six-step heuristic filter: (a) scans for explicit tags like '(Mobile)' or 'Phone:'; (b) strips LinkedIn URL slugs from the search buffer; (c) enforces a strict 10-to-15 digit constraint; (d) rejects 8-digit date ranges ($1950 \le Y_1, Y_2 \le 2050$); (e) rejects single-digit repetitions (e.g. 0000000000); and (f) excludes matches matching the candidate's profile slug.
• **Section Segmentation Heuristics:** Using regex lookaheads, the text corpus is partitioned into distinct semantic blocks: Headline, Summary/About, Experience, Education, and Skills.
• **Technical Entity Extraction:** Extracted tokens are cross-referenced against a curated dictionary of 60+ industry-standard technologies (e.g. Python, Docker, Kubernetes, React, FastAPI, CI/CD, System Design, PyTorch) to distinguish verified skills from general prose.
• **Readiness Scoring Mathematical Formulation:** Candidate resume readiness is computed via a bounded linear scoring model:

$$\text{Score}_{\text{raw}} = 70 + \min(|\text{Skills}| \times 1, 12) + \min(|\text{Experience}| \times 4, 10) + 5 \cdot \mathbb{I}_{\text{Edu}} + 4 \cdot \mathbb{I}_{\text{Cert}}$$

$$\text{Score}_{\text{final}} = \min(\max(\text{Score}_{\text{raw}}, 65), 98)$$

Where $\mathbb{I}_{\text{Edu}}$ and $\mathbb{I}_{\text{Cert}}$ are binary indicator flags (1 if present, 0 if absent), guaranteeing that candidate scores remain mathematically calibrated between 65% and 98%.

3.2 Feature 2: GitHub Repository & Code Hygiene Analyzer

Frontend User Workflow & Interaction Logic:

1. The candidate enters a GitHub username or full repository URL into the search field.
2. A real-time debounced autocomplete mechanism displays matching GitHub profiles with user avatars and bios.
3. Clicking a suggested user or submitting the form initiates an asynchronous analysis request.
4. The client transitions to a loading state while the backend executes concurrent repository audits.
5. The interface renders the Developer Score card, public repository statistics, stars, follower counts, and animated language progress bars.
6. A 'Start Test' CTA button is automatically configured for the candidate's highest-ranking programming language.

Backend Architecture, Data Pipeline, & Algorithmic Logic:

The GitHub Analyzer replaces superficial metric counting with deep static repository evaluations:
• **Input Normalization:** Input strings are sanitized to isolate canonical GitHub handles, handling SSH strings, protocol variants, and trailing .git tags.
• **Language Distribution Calculation:** The backend queries the GitHub REST API for up to 100 recent non-forked repositories, aggregating language bytes into a proportional distribution:
$$\text{Percentage}(L) = \max\left(\text{round}\left(\frac{\text{Bytes}(L)}{\sum \text{Bytes}} \times 100\right), 1\right)$$
• **Multi-Threaded File-Tree Inspection:** The engine identifies the top 8 most active repositories and inspects their root file trees concurrently using asynchronous I/O. The inspector verifies the physical existence of: automated testing directories or test files (has_tests), CI/CD workflow configurations in .github/workflows (has_workflows), containerization manifests like Dockerfile or docker-compose (has_docker), architectural documentation (has_readme), and open-source licenses (has_license).
• **Authentic Engineering Scoring Formulation:** The developer score is calculated from an empirical baseline adjusted for volume, community reach, and hygiene:

$$\text{Score}_{\text{base}} = 52 + \min(N_{\text{repos}}, 10) + \min(2 \cdot N_{\text{langs}}, 8) + \min\left(\left\lfloor \frac{\text{Stars}}{8} \right\rfloor, 10\right) + \min\left(\left\lfloor \frac{\text{Followers}}{10} \right\rfloor, 4\right)$$

$$\Delta_{\text{Hygiene}} = \begin{cases} -5 & \text{if } \text{TestRatio} = 0 \\ +8 & \text{if } \text{TestRatio} \ge 0.5 \end{cases} + \begin{cases} -5 & \text{if } \text{CIRatio} = 0 \\ +8 & \text{if } \text{CIRatio} \ge 0.3 \end{cases} + \begin{cases} +5 & \text{if } \text{ReadmeRatio} \ge 0.8 \\ -4 & \text{if } \text{ReadmeRatio} < 0.5 \end{cases} + (3 \text{ if } \text{LicenseRatio} \ge 0.5 \text{ else } 0)$$

$$\text{Score}_{\text{final}} = \min(\max(\text{Score}_{\text{base}} + \Delta_{\text{Hygiene}}, 45), 95)$$

This formula penalizes developers with un-tested or automated-pipeline-deficient codebases, regardless of star counts.

3.3 Feature 3: Anti-AI Code & Skill Authenticity Diagnostic Verifier

Frontend User Workflow & Interaction Logic:

1. The Verifier activates when GitHub or LinkedIn results exist, or when triggered directly from the language list.
2. The candidate reviews the auto-selected primary language or selects an alternate track from the dropdown.
3. Clicking 'Start Assessment' requests a custom diagnostic quiz and initializes an internal timer.
4. The candidate navigates 10 questions sequentially, reviewing code snippets and selecting answer options.
5. Upon final submission, the UI displays the Authenticity Scorecard, authenticity status label, diagnostic concept breakdown bars, and complete question-by-question reviews with detailed explanations.

Backend Architecture, Data Pipeline, & Algorithmic Logic:

The Verifier operates as an anti-vibe-coding diagnostic engine designed to detect whether claimed code competence represents genuine intuition:
• **Context-Linked Subject Auto-Resolution:** When invoked with language set to 'auto', the engine resolves the candidate's strongest verified programming language and extracts active repository names to contextualize question prompts.
• **Dual-Engine Dynamic Question Synthesis:** To prevent static answer leakage, questions are synthesized dynamically: (1) *Engine A (LLM Synthesis):* When configured with API keys, a generative model acting as a Principal Systems Engineer drafts 10 challenging language gotchas; (2) *Engine B (Intelligent Procedural Synthesizer):* An extensive algorithmic generator creates customized code snippets, alters numerical parameters, injects repository names, and applies Fisher-Yates shuffling to randomize answer key indices.
• **Forensic Language Gotchas Evaluated:** The engine tests deep runtime mechanics across languages: Python mutable default arguments and GIL contention; JavaScript event loop microtask vs. macrotask ordering and closure scoping traps; C pointer arithmetic precedence and struct alignment padding; C++ move semantics and virtual destructor invariants; Java string pool interning and finally block return overrides; Go goroutine closure capture and slice growth; and Rust borrow checker aliasing XOR mutability rules.
• **Authenticity Index Formulation:** Scoring directly computes the percentage of correct answers against total questions:

$$\text{Authenticity Index} = \left(\frac{\sum_{i=1}^{10} \mathbb{I}(\text{UserChoice}_i = \text{CorrectIndex}_i)}{10} \right) \times 100\%$$

Categorical verdicts are assigned according to strict threshold intervals:
• **Score ≥ 80% → Verified Authentic:** Demonstrates deep conceptual understanding of low-level execution mechanics.
• **50% ≤ Score < 80% → Likely AI-Augmented / Developing:** Possesses functional familiarity but remains vulnerable on compiler edge cases.
• **Score < 50% → Skill Gap Identified / Unverified Suspect AI:** High probability that repositories were assembled via generative LLM prompting without human mastery.

3.4 Feature 4: Unified Multi-Source Career Roadmap Synthesizer

Frontend User Workflow & Interaction Logic:

1. The Roadmap section displays status pills confirming which profile datasets have been successfully ingested.
2. The candidate clicks 'Build My Roadmap' (or the engine builds automatically upon dual analysis completion).
3. The interface renders the Combined Readiness Score card and four sequential sprint milestones.
4. Each milestone card details specific weekly focus areas, tangible deliverables, and an interactive task checklist.
5. Candidates can toggle task checkboxes to track progress, with state maintained during the browsing session.

Backend Architecture, Data Pipeline, & Algorithmic Logic:

The Roadmap engine executes cross-analyzer data synthesis to create repository-grounded learning sprints:
• **Multi-Source Ingestion & Gap Clustering:** The engine retrieves qualitative resume gaps (e.g. missing Docker, Kubernetes) and empirical GitHub hygiene gaps (e.g. absent test suites, missing CI/CD workflows). Skills are categorized into functional clusters: Testing, CI/CD, DevOps/Cloud, Typing, and Documentation.
• **Severity Ranking & Deduplication:** Gaps appearing across both sources are merged into unified priority items, weighted by market urgency (High, Medium, Low).
• **Repository-Grounded Sprint Generation:** Tasks are dynamically generated to reference the candidate's actual flagged repositories (e.g. 'Set up automated GitHub Actions in repo-name' rather than generic boilerplate advice).
• **Four-Sprint Progression Architecture:** Sprints are scheduled sequentially: Week 1 focuses on Unit Testing and Type Safety; Week 2 targets Backend Architecture and Modularity; Week 3 establishes Containerization and Automated CI/CD; and Week 4 solidifies Production Observability, Documentation, and Portfolio Alignment.

3.5 Feature 5: Context-Injected AI Career Coach Agent

Frontend User Workflow & Interaction Logic:

1. The candidate navigates to the Career Coach chat interface, noting the user context tag in the header.
2. The candidate clicks a pre-seeded suggestion pill or types a custom question into the input field.
3. User messages appear immediately in the chat stream alongside a typing indicator.
4. The assistant reply is rendered in real-time with structured markdown formatting.
5. The conversation transcript is maintained across interactions within the active session.

Backend Architecture, Data Pipeline, & Algorithmic Logic:

The Career Agent operates via a hybrid reasoning pipeline combining sub-second deterministic rule matching with ultra-fast LLM inference:
• **Context Ingestion & Memory Assembly:** Inbound messages are enriched with candidate data from the session store: detected skills, missing competencies, top GitHub programming languages, repository hygiene metrics, and previous conversation turns.
• **Deterministic Intent Matcher:** Predefined architectural queries (e.g. 'How does SkillPlus AI work?', 'Why is System Design important?') are matched via regex classifiers and answered instantly using structured technical knowledge templates, eliminating unnecessary API costs and latency.
• **Groq LPU Ultra-Fast Inference:** For complex technical inquiries, the engine dispatches requests to the Groq LPU inference endpoint running Llama-3.3-70B-versatile. The prompt injects the candidate's exact profile dossier and codebase metrics into the system prompt, instructing the model to ground its advice directly in their actual technical gaps.
• **Cloud Conversation Synchronization:** Completed message pairs are appended to in-memory history and mirrored asynchronously to Supabase cloud tables.

3.6 Feature 6: Real-Time Debounced GitHub Developer Autocomplete

Frontend User Workflow & Interaction Logic:

1. The candidate begins typing into the GitHub input field.
2. A debounced event listener waits 300ms after the last keystroke before dispatching a search query.
3. A floating suggestion dropdown displays matching candidate profiles, rendering avatars, usernames, and bios.
4. Selecting a suggestion populates the input field and automatically initiates repository analysis.

Backend Architecture, Data Pipeline, & Algorithmic Logic:

The Autocomplete engine balances responsive search against strict GitHub API rate limits:
• **In-Memory TTL Caching:** Search queries are cached in a dictionary with a 300-second Time-To-Live (TTL). Repeat searches return instantly without consuming API quotas.
• **Two-Stage Search Fallback:** The backend first queries the GitHub Search API using exact login matching ('q={clean}+in:login'). If zero results are returned, it executes a broader user search.
• **Asynchronous Profile Fan-Out:** For top search hits, the engine concurrently fetches user profile metadata via asynchronous HTTP requests to retrieve real display names and bios.
• **Prefix Sorting Heuristic:** Results are ordered prioritizing exact matches, followed by prefix matches, and lastly general substring matches.

RESEARCH METHODOLOGY SUMMARY: Mathematical and Algorithmic Consistency: Every feature in SkillPlus AI operates under deterministic mathematical models or grounded context-injected reasoning. No randomized or mock values exist within the system. The platform guarantees reproducible, evidence-based evaluations suitable for empirical research and enterprise talent assessment.

PART 4: Summary & Research Paper Integration Guide

When incorporating this data into academic research publications, the system components map directly to standard experimental software engineering and artificial intelligence publication structures:

• **System Methodology & Architecture:** Refer to Part 2 for the Unified Single-Origin Serverless ASGI topology and Multi-Tier Persistence engine.

• **Data Ingestion & Information Extraction:** Cite Section 3.1 for zero-disk PDF parsing and Contact Shield regex sanitization algorithms.

• **Static Code Analysis & Empirical Quality Metrics:** Cite Section 3.2 for multi-threaded AST-free repository file-tree inspections and the Engineering Hygiene scoring formulation.

• **Authenticity Verification & Anti-AI Diagnostics:** Cite Section 3.3 for the Dual-Engine question synthesis model and the Authenticity Index mathematical formulation.

• **Curriculum Generation & Adaptive Learning:** Cite Section 3.4 for the Cross-Analyzer gap clustering and 4-sprint progression algorithm.

• **Conversational Agents & Context Injection:** Cite Section 3.5 for Groq LPU low-latency inference and hybrid deterministic-generative reasoning.

Document Concluded — SkillPlus AI System Specification